Week 2 - Domain 1: SDLC Automation - AWS CodeDeploy

Study week: Oct 26 - Nov 1, 2026 **Exam:** AWS DOP-C02 (Domain 1 = 22% of the exam)

Section	What it covers
9.1 - 9.3	What CodeDeploy is, building blocks, agent, targets, revisions, appspec.yml for EC2
9.4 - 9.6	EC2 / on-premises lifecycle hooks, blue/green on EC2, deployment configurations, Auto Scaling
9.7 - 9.8	Lambda and ECS deployments (AppSpec, hooks, configurations)
9.9 - 9.12	Rollbacks, CloudWatch alarms, monitoring, IAM
9.13 - 9.16	Troubleshooting, exam traps, hands-on lab, check yourself

Part 9: AWS CodeDeploy

9.1 What CodeDeploy is

- **A managed deployment service** that installs a new application version and switches traffic to it on **EC2 / on-premises**, **Lambda** and **ECS**.
- Controls **how fast** a version rolls out, **checks health** at each step, and **rolls back automatically** on failure or when a CloudWatch alarm fires.
- Usually the **deploy stage of CodePipeline**, but works on its own (console, CLI, API).
- **No extra charge** for EC2, Lambda or ECS; on-premises is charged per instance update.

Part	What it is
Application	Container name for deployments; tied to one compute platform (EC2/On-premises, Lambda, ECS)
Deployment group	Targets + settings: instances / function / ECS service, deployment type, load balancer, alarms, rollback, triggers, service role
Deployment configuration	How fast the rollout goes (one at a time, 10% then the rest...)
Revision	The version deployed: code + appspec.yml (EC2), or just the AppSpec (Lambda, ECS)
Deployment	One run of a revision to a deployment group, with its own deployment ID
CodeDeploy agent	Software on EC2 / on-premises servers that does the work (not needed for Lambda / ECS)
Service role	IAM role CodeDeploy uses to work with EC2, Auto Scaling, load balancers, Lambda, ECS

Platform	Deployment types	How traffic moves	Hooks run as
EC2 / On-premises	In-place or blue/green (blue/green on EC2 only)	Instance by instance, or load balancer switches to new instances	Scripts on the instance
Lambda	Blue/green only (traffic shifting)	Alias shifts traffic from old version to new	Lambda functions
ECS	Blue/green only	Listener shifts traffic from old task set to new	Lambda functions

- **Exam point:** ECS **rolling updates** are built into ECS and don't use CodeDeploy; **ECS blue/green** uses CodeDeploy (ECS also gained its own blue/green option in 2025 - check the docs; the exam usually means CodeDeploy).

9.2 EC2 and on-premises: agent, targets, revisions

CodeDeploy agent:

- Install and keep it updated with **AWS Systems Manager** (State Manager association, `AWSCodeDeployAgent` package) - recommended. Or user data / AMI.

- The agent **connects out over HTTPS (443)** and asks for work - no inbound ports.
- Private subnets: **NAT gateway** or **VPC endpoints** for CodeDeploy (`codedeploy`, `codedeploy-commands-secure`) + **S3 gateway endpoint** for the revision.
- **Instance profile** must allow `s3:GetObject` on the revision bucket (+ `kms:Decrypt` for a customer-managed key).
- **Logs:** Linux `/var/log/aws/codedeploy-agent/codedeploy-agent.log` and per-deployment script output under `/opt/codedeploy-agent/deployment-root/`; Windows `C:\ProgramData\Amazon\CodeDeploy\log`. Ship to CloudWatch Logs with the CloudWatch agent.

On-premises servers:

- **Register** each server, then **tag** it. Preferred: an **IAM session ARN** (temporary credentials, e.g. STS / IAM Roles Anywhere). Older: IAM user access keys in `codedeploy.onpremises.yml`.
- **Blue/green is not supported on-premises** - in-place only.

Targets:

- **EC2 tags:** up to **3 tag groups** - an instance must match **every** group (AND); inside a group, **any** tag (OR).
- **Auto Scaling group names**; **on-premises instance tags**.
- No matching instances = deployment fails (check tags first).

Revisions:

- **S3** (`.zip`, `.tar`, `.tgz`; versioned bucket) or **GitHub** (a commit).
- `appspec.yml` **must be at the root of the revision.**

9.3 appspec.yml for EC2 / on-premises

```
version: 0.0
os: linux                       # or windows
files:
  - source: /app
    destination: /var/www/myapp
file_exists_behavior: OVERWRITE # DISALLOW (default) / OVERWRITE / RETAIN
permissions:
  - object: /var/www/myapp
    owner: webapp
    group: webapp
    mode: 755
    type: [directory, file]
hooks:
  ApplicationStop:
    - location: scripts/stop_server.sh
      timeout: 60
      runas: root
  BeforeInstall:
    - location: scripts/install_dependencies.sh
      timeout: 300
  AfterInstall:
    - location: scripts/configure.sh
  ApplicationStart:
    - location: scripts/start_server.sh
      timeout: 120
  ValidateService:
    - location: scripts/health_check.sh
      timeout: 120
```

Section	Meaning
version	Always 0.0
os	linux or windows
files	What to copy from the revision and where
file_exists_behavior	A file already exists that the previous deployment didn't put there: DISALLOW (default) fails; OVERWRITE replaces; RETAIN keeps the existing file (also settable per deployment)
permissions	Owner, group, mode for copied files (Linux)
hooks	Scripts per step: location, timeout (max 3600 seconds per script), runas (Linux user)

- **Exit codes decide success:** non-zero exit or timeout = that step fails = the deployment on that instance fails.

9.4 Lifecycle hooks for EC2 / on-premises

In-place order:

```
(with a load balancer)  BeforeBlockTraffic > BlockTraffic > AfterBlockTraffic
ApplicationStop > DownloadBundle > BeforeInstall > Install > AfterInstall
  > ApplicationStart > ValidateService
(with a load balancer)  BeforeAllowTraffic > AllowTraffic > AfterAllowTraffic
```

Event	Script?	Typical use
BeforeBlockTraffic	Yes	Tasks before leaving the load balancer
BlockTraffic	No (CodeDeploy)	Removes the instance from the load balancer (draining)
AfterBlockTraffic	Yes	Tasks once no traffic reaches the instance
ApplicationStop	Yes	Stop the running application gracefully
DownloadBundle	No	Agent copies the revision to a temporary folder
BeforeInstall	Yes	Decrypt files, back up current version, install dependencies
Install	No	Agent copies files to their destinations
AfterInstall	Yes	Configure the app, change permissions
ApplicationStart	Yes	Start the services
ValidateService	Yes	Check the deployment worked (health check / smoke test) - exit non-zero to fail
BeforeAllowTraffic	Yes	Tasks before rejoining the load balancer
AllowTraffic	No	Puts the instance back - must pass load balancer health checks
AfterAllowTraffic	Yes	Tasks after serving traffic again

ApplicationStop detail (often tested):

- `ApplicationStop` runs the script from the **previous successful revision** (new files aren't downloaded yet).
- If the old stop script is broken, **every new deployment fails at ApplicationStop** - even a fixed revision.
- **Fix:** deploy with `--ignore-application-stop-failures` (console: "Don't fail the deployment if the ApplicationStop step fails"), or clean up the old revision info on the instance.
- On the **first** deployment to an instance there is no previous revision, so ApplicationStop does nothing.

Blue/green on EC2:

- **Requires a load balancer** (Classic, Application or Network Load Balancer).
- Green instances: **copy the Auto Scaling group** automatically, or **choose instances** by tags.
- Steps: deploy to green (ApplicationStop ... ValidateService, then BeforeAllowTraffic / AllowTraffic / AfterAllowTraffic) > **move traffic immediately** or **wait for you to continue** (within a time limit; otherwise it stops) > blue runs BeforeBlockTraffic / BlockTraffic / AfterBlockTraffic > **terminate blue after a wait time, or keep it** (quick rollback).
- Benefits: no downtime, fast rollback. Cost: two fleets for a while.

9.5 Deployment configurations (EC2 / on-premises)

Configuration	Behaviour	Deployment succeeds if
CodeDeployDefault .AllAtOnce	Every instance at once (fastest, downtime risk)	At least one instance succeeds
CodeDeployDefault .HalfAtATime	Up to half at a time	At least half succeed
CodeDeployDefault .OneAtATime	One at a time (slowest, safest)	Every instance succeeds - except the last one is allowed to fail

Configuration	Behaviour	Deployment succeeds if
Custom	Minimum healthy hosts as HOST_COUNT (number) or FLEET_PERCENT (%)	Fails if fewer stay healthy
Zonal configuration	One Availability Zone at a time, with a monitoring wait after each zone (and after the first)	Limits damage to one AZ

- **Minimum healthy hosts** decides how many instances can be taken out at once; too many failures = it stops (HEALTH_CONSTRAINTS).
- Blue/green on EC2 uses these for the deployment to green and for moving traffic in batches.

9.6 Auto Scaling groups with CodeDeploy

- With an Auto Scaling group in the deployment group, CodeDeploy adds a **lifecycle hook**: **new scale-out instances automatically get the last successful revision** before serving traffic.
- **Scale-out during a deployment:** new instances may get the old revision; CodeDeploy starts a **follow-up deployment**. Best practice: **pause scaling** during deployments, or rely on the follow-up.
- **Scale-in termination hooks:** CodeDeploy can run ApplicationStop / BeforeBlockTraffic on instances being removed (graceful shutdown).
- **Trap:** if the lifecycle-hook deployment on new instances fails (bad script), the group keeps launching and terminating instances.

9.7 Lambda deployments

```
version: 0.0
Resources:
  - MyFunction:
      Type: AWS::Lambda::Function
      Properties:
        Name: payments-fn
        Alias: live
        CurrentVersion: "7"
        TargetVersion: "8"
Hooks:
  - BeforeAllowTraffic: PreTrafficCheckFn
  - AfterAllowTraffic: PostTrafficCheckFn
```

- CodeDeploy moves the **alias** from CurrentVersion to TargetVersion step by step.
- **Hook order:** Start > **BeforeAllowTraffic** > AllowTraffic > **AfterAllowTraffic** > End.
- **Hooks are Lambda functions** that must call `codedeploy:PutLifecycleEventHookExecutionStatus` (Succeeded / Failed); otherwise the hook **fails after 1 hour**.

Type	Built-in configurations (prefix CodeDeployDefault.)	Behaviour
Canary	LambdaCanary10Percent5Minutes / 10Minutes / 15Minutes / 30Minutes	10%, wait, then all the rest at once
Linear	LambdaLinear10PercentEvery1Minute / 2Minutes / 3Minutes / 10Minutes	Another 10% every N minutes to 100%
All at once	LambdaAllAtOnce	100% immediately
Custom	Your own percentage and interval	

With AWS SAM (usual exam answer):

```
Resources:
  PaymentsFn:
    Type: AWS::Serverless::Function
    Properties:
      AutoPublishAlias: live
      DeploymentPreference:
        Type: Canary10Percent5Minutes
        Alarms:
          - !Ref ErrorsAlarm
        Hooks:
          PreTraffic: !Ref PreTrafficCheckFn
          PostTraffic: !Ref PostTrafficCheckFn
```

- **AutoPublishAlias** publishes a new version each deploy and points the alias at it; **DeploymentPreference** creates the CodeDeploy application and deployment group.

9.8 ECS deployments (blue/green)

Requirements:

- ECS service with `deploymentController: CODE_DEPLOY`.
- **ALB or NLB** with **two target groups** (blue and green).
- A **production listener** and an optional **test listener** (e.g. port 8080) to test new tasks before real users.
- CodeDeploy creates a **replacement task set**, moves traffic, then removes the **original task set**.

```
version: 0.0
Resources:
  - TargetService:
      Type: AWS::ECS::Service
      Properties:
        TaskDefinition: "arn:aws:ecs:us-east-1:111122223333:task-definition/web:12"
        LoadBalancerInfo:
          ContainerName: "web"
          ContainerPort: 8080
        PlatformVersion: "LATEST"
Hooks:
  - BeforeInstall: "PreInstallFn"
  - AfterInstall: "PostInstallFn"
  - AfterAllowTestTraffic: "RunTestsOnTestListenerFn"
  - BeforeAllowTraffic: "PreProdTrafficFn"
  - AfterAllowTraffic: "SmokeTestFn"
```

ECS hook order:

```
Start > BeforeInstall > Install > AfterInstall > AllowTestTraffic > AfterAllowTestTraffic
  > BeforeAllowTraffic > AllowTraffic > AfterAllowTraffic > End
```

Hook	Typical use
BeforeInstall	Before the replacement task set is created
AfterInstall	After it's created, before any traffic
AfterAllowTestTraffic	Run tests through the test listener against new tasks
BeforeAllowTraffic	Last check before production traffic moves
AfterAllowTraffic	Smoke tests once production traffic is on new tasks

- Hooks are **Lambda functions** calling `PutLifecycleEventHookExecutionStatus`.

Configuration (prefix CodeDeployDefault.)	Behaviour
ECSAllAtOnce	All traffic at once
ECSCanary10Percent5Minutes / ECSCanary10Percent15Minutes	10%, wait, then the rest
ECSLinear10PercentEvery1Minutes / ECSLinear10PercentEvery3Minutes	Another 10% every N minutes
Custom	Your own percentage and interval

- **Original task set kept for a wait time** after traffic moves - instant rollback during that time.

- **CodePipeline CodeDeployToECS** needs `taskdef.json`, `appspec.yaml`, `imageDetail.json` (`<IMAGE1_NAME>` placeholder replaced with the new image).
- ECS service role needs `iam:PassRole` for the task roles.

9.9 Rollbacks

Type	How
Automatic, on failure	Deployment group: "Roll back when a deployment fails"
Automatic, on alarm	"Roll back when alarm thresholds are met" - needs alarms on the deployment group
Manual	Stop a deployment and roll back, or redeploy the previous revision
Per deployment	Override the group's rollback settings when starting a deployment

- **EC2 / on-premises:** a **new deployment** (**new deployment ID**) of the **last successful revision**; in-place redeploys it to the instances (all hooks run again).
- **Blue/green EC2:** traffic goes back to the original instances (if not terminated).
- **Lambda:** alias moves back to the original version immediately.
- **ECS:** traffic moves back to the original task set immediately.
- **Trap:** no last known good revision (e.g. the very first deployment fails) = nothing to roll back to.

9.10 CloudWatch alarms

- **Up to 10 CloudWatch alarms** per deployment group.
- An alarm in **ALARM during a deployment stops it**, and **rolls back** if that option is on.
- Options: **ignore alarm configuration** (e.g. emergency fix); **continue if alarm status can't be read** from CloudWatch.
- Typical alarms: Lambda Errors / Throttles (new version / alias), ALB HTTPCode_Target_5XX_Count, TargetResponseTime, application metrics, **Synthetics canary failures**.
- **Exam pattern:** shift Lambda traffic gradually, roll back if errors rise = **canary / linear + alarm on the deployment group + rollback on alarm**.

9.11 Monitoring and notifications

- **Deployment group triggers** > **SNS** for deployment and instance events (up to 10 triggers).
- **EventBridge:** "CodeDeploy Deployment State-change Notification", "CodeDeploy Instance State-change Notification" > Lambda, Step Functions, tickets.
- **Notification rules** > SNS / Slack / Teams (Chatbot), no code.
- **CloudTrail** for API calls; **agent and script logs** on instances (ship to CloudWatch Logs).

9.12 IAM

Role	Used by	Needs
Service role (EC2 / on-premises)	CodeDeploy	AWSCodeDeployRole (EC2, Auto Scaling, load balancers, tags, SNS)
Service role (Lambda)	CodeDeploy	AWSCodeDeployRoleForLambda (update alias, invoke hooks)
Service role (ECS)	CodeDeploy	AWSCodeDeployRoleForECS + iam:PassRole on task roles
Instance profile	Agent on EC2	s3:GetObject on the revision bucket (+ kms:Decrypt)
On-premises identity	Agent on your server	IAM session ARN (preferred) or IAM user
Hook function role	Lambda / ECS hook functions	codedeploy:PutLifecycleEventHookExecutionStatus + what the tests need

9.13 Troubleshooting

Symptom	Likely cause and fix
Deployment never starts on an instance	Agent not installed / running, no outbound 443 (NAT or endpoints), or instance profile missing
Fails at DownloadBundle	Instance profile can't read the S3 revision / KMS key, or no S3 route / endpoint
AppSpec file not found	appspec.yml not at the root of the bundle
Every deployment fails at ApplicationStop	Previous revision's stop script is broken - use --ignore-application-stop-failures
Fails at Install: file already exists	file_exists_behavior DISALLOW (default) - use OVERWRITE or RETAIN
A script step fails or times out	Non-zero exit or longer than timeout (max 3600 s) - read logs under deployment-root
HEALTH_CONSTRAINTS	Too many instances failed for minimum healthy hosts - fix the cause or choose a suitable configuration
No instances found	Tags / tag groups / Auto Scaling group name don't match
AllowTraffic fails	Instances fail load balancer health checks (port, path, security group)
Lambda / ECS hook fails after about 1 hour	Hook function never called PutLifecycleEventHookExecutionStatus
ECS deployment won't start	Service not using the CODE_DEPLOY controller, target groups / listener missing, or iam:PassRole missing
New Auto Scaling instances keep being replaced	Lifecycle-hook deployment on new instances failing
Deployment stopped with no failed instances	A CloudWatch alarm went into ALARM - stopped or rolled back

9.14 Exam traps: quick table

Scenario	Answer
Install / update the agent on many instances	Systems Manager (State Manager, AWSCodeDeployAgent)
Agent in private subnets can't reach CodeDeploy	NAT gateway or CodeDeploy VPC endpoints + S3 gateway endpoint
EC2 in-place hook order	ApplicationStop > BeforeInstall > AfterInstall > ApplicationStart > ValidateService
Check the deployment really worked on EC2	ValidateService script that exits non-zero on failure
Every new deployment fails at ApplicationStop	--ignore-application-stop-failures
Existing files block the install	file_exists_behavior OVERWRITE (or RETAIN)
No capacity loss and fast rollback on EC2	Blue/green (copy Auto Scaling group + load balancer)
On-premises and blue/green	Not supported - in-place only
Keep at least 75% of instances serving	Custom config: minimum healthy hosts FLEET_PERCENT 75
Limit damage to one Availability Zone	Zonal deployment configuration
New Auto Scaling instances must get the current version	Auto Scaling group in the deployment group (lifecycle hook)
Lambda: 10% for 10 minutes, then everything	LambdaCanary10Percent10Minutes
Lambda: another 10% every minute	LambdaLinear10PercentEvery1Minute
Serverless traffic shifting, least effort	SAM AutoPublishAlias + DeploymentPreference
Test new ECS tasks before production traffic	Test listener + AfterAllowTestTraffic hook
ECS blue/green requirements	CODE_DEPLOY controller, two target groups, prod (+ test) listener
Lambda / ECS hook hangs then fails	Doesn't call PutLifecycleEventHookExecutionStatus
Roll back automatically when errors rise	Alarm on the deployment group + rollback on alarm
What a rollback creates on EC2	A new deployment of the last good revision (new deployment ID)

Scenario	Answer
Deploy now although an alarm is firing	Ignore alarm configuration option
Email the team when a deployment fails	Deployment group trigger > SNS (or notification rule / EventBridge)
ECS rolling update	ECS's own deployment controller, not CodeDeploy

9.15 Hands-on lab (about 3 hours)

1. **EC2 in-place:** 2 Amazon Linux instances tagged `App=dop-lab`, instance profile with S3 read, agent installed by **Systems Manager**.
2. Revision with appspec.yml hooks ApplicationStop, BeforeInstall, ApplicationStart, ValidateService (`curl --fail localhost`); zip and upload to S3.
3. Application + deployment group; deploy with **OneAtATime**.
4. Break it: ValidateService exits 1 - read logs under `/opt/codedeploy-agent/deployment-root/`. Break the stop script, deploy, then deploy a fixed revision - it fails at ApplicationStop; redeploy with `--ignore-application-stop-failures`.
5. Rollback: turn on "roll back when a deployment fails", deploy a bad revision, see the new rollback deployment.
6. **Lambda with SAM:** AutoPublishAlias + Canary10Percent5Minutes + an Errors alarm; deploy an erroring version and watch the automatic rollback.
7. Optional **ECS blue/green:** Fargate service with CODE_DEPLOY controller, two target groups, test listener, AfterAllowTestTraffic hook, ECSLinear10PercentEvery1Minutes.
8. Clean up: instances, Auto Scaling groups, load balancer, SAM stack, ECS service, S3 revisions.

9.16 Check yourself

1. List the EC2 in-place hooks you can script, in order, with no load balancer.
2. Every new deployment fails at ApplicationStop, even with a fixed revision. Why, and what is the fix?
3. Difference between LambdaCanary10Percent5Minutes and LambdaLinear10PercentEvery1Minute?
4. Which ECS hook runs tests against new tasks before production traffic?
5. A Lambda hook runs its tests correctly, but the deployment fails after an hour. Why?
6. On EC2 in-place, what does an automatic rollback actually do?

Answers:

1. ApplicationStop > BeforeInstall > AfterInstall > ApplicationStart > ValidateService.
2. ApplicationStop runs the **previous** revision's broken script - deploy with **--ignore-application-stop-failures**.
3. Canary: **10%, wait 5 minutes, then everything**. Linear: **another 10% every minute** to 100%.
4. **AfterAllowTestTraffic** (through the test listener).
5. The function never called **PutLifecycleEventHookExecutionStatus**.
6. Creates a **new deployment** of the **last successful revision** and redeploys it (all hooks run again).